Synechron L2 – Tricky Questions & Spoken Answers

Prepared directly from the attached interview-preparation document. The original question order, terminology and answers are preserved.

Yes. For tomorrow's Synechron L2, I would expect some **cross-domain and “how exactly?” questions** rather than only standard definitions. The interviewer may deliberately test whether your DevOps/SRE/platform experience is genuinely hands-on.

Here are the tricky ones I would prepare.

■ 1. “You say you are a Platform Engineer. What platform did you actually build?”

This is potentially one of the most important questions.

Answer:

“My platform engineering experience has been more around operating and improving shared cloud and Kubernetes platforms rather than building a complete internal developer platform from scratch. I worked on standardising deployment, infrastructure automation, observability, access, monitoring and operational processes. For example, using Terraform, Kubernetes, CI/CD and monitoring to provide repeatable deployment and operational capabilities. I would not claim that I built an entire IDP product end-to-end, but the engineering practices are directly aligned with platform engineering.”

■ 2. “What's the difference between DevOps and SRE?”

Answer:

“DevOps is a broader engineering culture and delivery model that brings development and operations together through automation and collaboration. SRE applies software engineering principles to operations, with reliability measured through SLIs, SLOs and error budgets. DevOps focuses strongly on delivery flow, while SRE puts stronger emphasis on measurable reliability and operational excellence. In practice they overlap heavily.”

■ 3. “What's the difference between Platform Engineering and SRE?”

Answer:

“Platform Engineering primarily builds reusable capabilities and golden paths for engineering teams, while SRE primarily focuses on reliability of services and systems. A platform team might provide Kubernetes, CI/CD templates, Terraform modules and observability as a service. SRE would use those capabilities to operate services reliably and define SLOs, manage incidents and reduce toil.”

■ 4. “What exactly is a golden path?”

Answer:

“A golden path is a recommended, standardised way for developers to perform a common engineering task. For example, creating a service could automatically provision the repository, CI/CD pipeline, container configuration, Kubernetes deployment, monitoring dashboard and required security controls. The goal is to make the secure and operationally supported path the easiest path.”

■ 5. “What is toil? Is every manual activity toil?”

Answer:

“No. Toil is manual, repetitive, automatable operational work that provides little enduring value and tends to grow with the system. An activity can be manual but still be valuable, such as an emergency investigation or architectural decision. I look for repetitive operational work that consumes engineering time and automate it where the risk-benefit makes sense.”

Then give your **Rundeck/cache-clear example**.

■ 6. “Your application is healthy according to Kubernetes. Users say it is down. Who is right?”

This is a very good SRE question.

Answer:

“Both measurements can be technically correct. Kubernetes health only tells me about the health signal represented by the probes. Users could still be experiencing DNS, ingress, authentication, dependency, database or business-transaction failures. I would start from the user journey and correlate it with application metrics, ingress metrics, logs, traces and dependency health.”

■ 7. “Readiness probe is passing, but users are getting 500s. Why?”

Possible causes:

• Probe only checks process availability.

• Application dependency is failing.

• Specific API endpoint is broken.

• Database unavailable.

• Authentication problem.

• Ingress/routing issue.

• Business logic failure.

Answer:

“A readiness probe doesn't automatically prove that every business transaction works. I would check the actual failing endpoint, ingress or load-balancer metrics, application logs, dependency health and traces. If appropriate, I would improve the readiness or synthetic monitoring so it represents meaningful service health without making the probe itself dependency-heavy.”

■ 8. “Why shouldn't you put a database health check directly into a liveness probe?”

Answer:

“Because if the database temporarily fails, every application pod could become unhealthy and restart simultaneously. That can turn a dependency failure into a larger outage. I would generally keep liveness focused on whether the process is capable of recovering, while readiness can determine whether the pod should receive traffic.”

Excellent L2 question.

■ 9. “What happens if Kubernetes API server is down?”

Answer:

“Existing workloads can generally continue running on worker nodes because kubelet and the container runtime continue managing existing pods. However, cluster management operations such as scheduling new pods, scaling through the control plane and Kubernetes API interactions will be affected. In EKS, the control plane is AWS-managed, so I would investigate the EKS service/control-plane health and AWS status while protecting existing workloads.”

■ 10. “What happens if a worker node dies?”

Answer:

“The workloads on that node become unavailable. Kubernetes detects the node condition and, for workloads managed by a controller such as a Deployment, replacement pods can be scheduled on healthy nodes if capacity is available. I would check replica distribution, topology spread, PDBs, node capacity and whether the service is exposed through a healthy load balancer.”

■ 11. “Why would a Kubernetes pod have an IP but still not connect to another service?”

Think through the complete network path:

Potential problems:

• NetworkPolicy

• wrong Service selector

• no endpoints

• wrong targetPort

• application listening on wrong interface/port

• DNS

• security rules

• CNI issue

Answer:

“I wouldn't assume that having a pod IP proves connectivity. I would check DNS resolution, Service and Endpoints, targetPort, NetworkPolicies, pod listening ports and CNI/network behaviour. I would test from the source pod using curl or nc and follow the network path systematically.”

■ 12. “Service exists, but traffic isn't reaching pods. What do you check?”

Say this sequence:

“Service selector → Endpoints/EndpointSlices → targetPort → Pod readiness → NetworkPolicy → application listening port → kube-proxy/CNI → ingress or load balancer.”

This sounds much stronger than simply saying “check the Service.”

■ 13. “Why does EKS run out of IP addresses even when EC2 has enough CPU?”

Answer:

“Because AWS VPC CNI assigns VPC IP addresses to pods. CPU and memory can be available while the subnet has insufficient IP capacity or the node has reached its ENI/IP allocation limits. I would check subnet free IPs, ENI allocation, pod density, instance type limits and CNI configuration.”

This is **very relevant after your L1 CIDR discussion**.

■ 14. “Would you choose /21 for 1,500 IPs?”

Don't simply say yes.

Answer:

“A /21 provides 2,048 total IPv4 addresses, so mathematically it covers 1,500. But I wouldn't select it based only on that calculation. In AWS I account for the five reserved addresses per subnet, EKS node and pod requirements, ENI limits and future growth. If 1,500 is current consumption rather than the final requirement, I would normally provide additional headroom.”

■ 15. “Route 53 or Global Accelerator for active-active?”

Answer:

“Both can provide health-based failover, but they work differently. Route 53 provides DNS-based routing and health checks. Global Accelerator provides static anycast IPs and routes traffic over the AWS global network toward healthy regional endpoints. For fast application traffic failover where static IPs and global network routing are useful, Global Accelerator is an option.”

If they say:

Don't agree.

“No, Route 53 can perform automatic health-based DNS failover when health checks and routing policies are configured. Global Accelerator provides a different mechanism with network-level routing and static anycast IPs.”

■ 16. “What is the difference between RTO and RPO?”

Answer:

“RTO is how quickly we need to restore the service after a disruption. RPO is how much data loss, measured in time, the business can tolerate. For example, an RTO of one hour means the service should recover within an hour, while an RPO of 15 minutes means we should limit recoverable data loss to approximately 15 minutes.”

■ 17. “Active-active means zero downtime. Correct?”

Answer:

“Not necessarily. Active-active reduces the impact of a regional failure because multiple sites are serving traffic, but it doesn't guarantee zero downtime. DNS or traffic-routing convergence, application defects, dependency failures and data replication issues can still cause impact. The architecture should be validated through failover testing against defined RTO and RPO.”

■ 18. “Terraform apply is running and someone changes AWS manually. What happens?”

Answer:

“It depends on when the change occurs and whether Terraform refreshes the state before applying. Terraform's desired configuration remains the source of truth. The next plan should expose drift. I would avoid simultaneous manual and Terraform changes because they create race conditions and unpredictable outcomes. For production, changes should go through the controlled IaC process.”

■ 19. “Terraform state gets deleted. What happens?”

Answer:

“The real infrastructure doesn't automatically disappear, but Terraform loses its state mapping between configuration and resources. Recovery depends on the backend and backups. That's why production Terraform should use a remote backend with versioning, access control and locking. If necessary, resources can be re-imported into state, but that needs careful reconciliation.”

■ 20. “Terraform plan shows it will destroy production resources. What do you do?”

Answer:

“I stop and investigate rather than applying it. I identify why Terraform thinks the resource must be replaced, review state, configuration changes, provider behaviour and lifecycle settings, and determine the impact. For a production resource, destructive changes require explicit review and approval.”

■ 21. “Can you use `terraform destroy` in production?”

Don't just say “no.”

“Technically Terraform can do it, but operationally I would not treat production destruction as a normal workflow. Production infrastructure should have strong access controls, approvals and safeguards. Any destructive operation would require explicit change governance and a validated recovery plan.”

■ 22. “What if Terraform state is locked?”

Answer:

“First I determine whether another legitimate Terraform operation is running. I don't force-unlock blindly. If the process has genuinely failed and the lock is stale, I verify that no active operation exists and then use the approved force-unlock procedure.”

■ 23. “How do you prove a release was successful?”

This is especially important for Synechron.

Answer:

“I don't consider deployment completion alone as success. I validate deployment status, pod or instance health, application logs, error rate, latency, infrastructure saturation, dependency health and business smoke tests. I compare key metrics with the pre-release baseline and capture evidence against the change record.”

■ 24. “Deployment succeeded but release failed. How?”

Very good trick question.

“Deployment success only means the deployment mechanism completed. The application can still fail functionally or operationally. For example, pods may be running but returning 500s, latency may violate the SLO, a database migration may be incompatible, or a downstream dependency may fail. That's why post-deployment validation is essential.”

■ 25. “Who makes the Go/No-Go decision?”

Don't say “I do.”

“It depends on the organisation's governance model. I coordinate the readiness assessment and present the technical status, risks, dependencies, approvals and rollback readiness. The authorised release or business owner makes the formal decision where governance requires it. I would never bypass an approval gate based solely on my own judgement.”

■ 26. “What if the application team says rollback isn't possible?”

“Then I don't accept ‘rollback isn't possible’ as the end of the discussion. I identify why—database migration, irreversible data change, external dependency, schema incompatibility—and define an alternative recovery strategy such as forward fix, traffic shift, feature flag, compensating change or restore procedure. The recovery strategy must be agreed before production.”

■ 27. “What is Change Failure Rate?”

Answer:

“It is the percentage of deployments or changes that result in a failure requiring remediation, rollback, hotfix or causing an incident, depending on the organisation's exact definition. I use it with deployment frequency, lead time and recovery time to understand delivery performance without optimising one metric in isolation.”

■ 28. “Your deployment frequency is high but incidents are increasing. Is that good DevOps?”

Answer:

“Not by itself. High deployment frequency is useful only when delivery remains safe and reliable. I would look at change failure rate, escaped defects, incident volume, MTTR and SLO performance. The objective is sustainable delivery speed with reliability, not deployment frequency in isolation.”

■ 29. “What would you automate first in a platform?”

A strong answer:

“I would start with high-volume, repetitive and standardisable activities that have clear inputs and outputs. Typically service scaffolding, CI/CD templates, infrastructure provisioning, environment configuration, secrets integration, observability setup and standard deployment validation. I would prioritise based on toil reduction, risk reduction and developer impact.”

■ 30. “What would you do in your first 90 days?”

This is **very likely for an L2/senior candidate**.

First 30 days

“Understand the architecture, release process, platform, tooling, stakeholders, production incidents and current pain points.”

30–60 days

“Identify reliability and delivery bottlenecks, improve monitoring/runbooks, reduce obvious manual toil and standardise recurring processes.”

60–90 days

“Deliver measurable improvements such as deployment reliability, automation, alert quality, release predictability and operational readiness.”

■■ Three questions where you should NOT bluff

Given your L1, these are particularly important.

Service Mesh

If asked:

“Have you implemented Istio?”

Say:

“I understand Istio and service-mesh concepts such as mTLS, traffic management, retries, circuit breaking and telemetry, but I haven't personally owned an Istio implementation end-to-end. My hands-on experience has been stronger in Kubernetes networking, ingress, services, NetworkPolicies and production operations.”

That's much safer than inventing experience.

Power BI

If asked:

“Have you integrated Synapse with Power BI?”

Say:

“I have worked with Synapse operationally and have seen Power BI used for reporting, but I haven't personally owned the Power BI integration end-to-end.”

Banking

If asked:

“Which banking application did you support?”

Don't invent one.

Say:

“My direct experience is primarily enterprise production environments rather than owning a specific banking application. However, I have worked with controlled production changes, incident management, audit-oriented processes, security controls and release governance, which are directly applicable to a regulated banking environment.”

■ The “How exactly?” trap

This is probably what I'd prepare for most.

They may ask:

“How do you monitor Kubernetes?”

Don't answer:

“Prometheus and Grafana.”

Instead:

“Prometheus scrapes metrics from the cluster and applications, Grafana visualises them, and Alertmanager handles alert routing. I monitor node and pod CPU/memory, restarts, resource saturation, API/server health, deployment availability, request rate, latency and errors. For troubleshooting I correlate those metrics with Kubernetes events, container logs and application logs, and traces where available.”

They may then ask:

Be ready with:

CPU → Memory → Restarts → OOMKilled → Pod availability → Replica availability → Request rate → Error rate → p95/p99 latency → Saturation → Network → Disk → API-server health

---

■ One final thing I'd prepare tonight

The interviewer may deliberately combine **all four areas in one scenario**:

****“We have a banking application running on EKS. A new release is deployed through Terraform/Jenkins. Deployment succeeds, but customers start seeing intermittent 500 errors. As an SRE/Platform Engineer, what do you do?”****

Your answer should flow:

A strong spoken answer:

“First I establish customer impact—error rate, affected endpoints, regions and severity. Then I correlate the timing with the deployment and check ALB or ingress metrics, application logs, pod status, Kubernetes events, latency and dependency health. I verify whether the new pods are actually healthy and whether the issue is application, configuration, database or networking related. If the deployment is clearly correlated and rollback criteria are met, I stop the rollout and revert to the last known-good version. I then validate the error rate, latency and business transaction through smoke tests and monitoring. Throughout the incident I keep stakeholders informed and maintain the change and incident records. After recovery, I complete RCA and add preventive controls such as better deployment validation, canarying, automated rollback or additional observability.”

For tomorrow, I would spend the most time on **EKS networking/CIDR, Kubernetes troubleshooting, Terraform, Platform Engineering concepts, SRE scenarios, and banking release governance** rather than trying to memorise dozens of generic definitions.